深圳易新泰科技有限公司YiXinTai

RTOS SPILCD
调试指南

版本号: 1.1
发布日期: 2023.12.10

版本历史

版本号	日期	制/修订人	内容描述
1.0	2022.10.10	AWA1683	初始版本
1.1	2023.12.10	AWA1799	重新排版

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 概述 1

1.1 文档简介 . . . 1
1.2 目标读者 . . . 1
1.3 适用范围 . . . 1
1.4 文档约定 . . . 2
1.4.1 标志说明 . . . 2
1.4.2 地址与数据描述方法约定 . . . 2
1.4.3 数值单位约定 . . . 2
1.5 相关术语介绍 . . . 3
1.5.1 硬件术语 . . . 3
1.5.2 软件术语 . . . 3

2 模块介绍 4

2.1 基础说明 . . . 4
2.2 接口说明 . . . 4
2.3 快速评估 . . . 7

3 编写屏驱动 9

3.1 编写步骤 . . . 9
3.2 开关屏流程函数解析 . . . 12
3.2.1 LCD_power_on . . . 13
3.2.2 LCD_panel_init . . . 13
3.2.3 lcd_fb_black_screen . . . 13
3.3 常用驱动函数解析 . . . 14
3.3.1 LCD_OPEN_FUNC . . . 14
3.3.2 sunxi_lcd_delay_ms . . . 15
3.3.3 sunxi_lcd_backlight_enable . . . 15
3.3.4 sunxi_lcd_backlight_disable . . . 15
3.3.5 sunxi_lcd_pwm_enable . . . 15
3.3.6 sunxi_lcd_pwm_disable . . . 16
3.3.7 sunxi_lcd_power_enable . . . 16
3.3.8 sunxi_lcd_power_disable . . . 17
3.3.9 sunxi_lcd_cmd_write . . . 17
3.3.10 sunxi_lcd_para_write . . . 17
3.3.11 sunxi_lcd_gpio_set_value . . . 18
3.3.12 sunxi_lcd_gpio_set_direction . . . 18

4 模块配置介绍 19

4.1 板级显示配置说明 . . . 19

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.1.1 使用 sys_config.fex 的方式进行配置 . . . 19
4.1.2 使用显示私有方式进行配置 . . . 20
4.2 板级显示配置参数详解 . . . 22
4.2.1 lcd_driver_name . . . 22
4.2.2 lcd_model_name . . . 22
4.2.3 lcd_if . . . 22
4.2.4 lcd_dbi_if . . . 22
4.2.5 lcd_dbi_fmt . . . 23
4.2.6 lcd_dbi_te . . . 23
4.2.7 lcd_dbi_clk_mode . . . 23
4.3 lcd_rgb_order . . . 24
4.3.1 针对 rgb565 格式额外说明 . . . 24
4.4 lcd_x . . . 25
4.5 lcd_y . . . 25
4.6 lcd_data_speed . . . 25
4.7 lcd_fps . . . 25
4.8 lcd_pwm_used . . . 25
4.9 lcd_pwm_ch . . . 26
4.10 lcd_pwm_freq . . . 26
4.11 lcd_pwm_pol . . . 26
4.12 lcd_pwm_max_limit . . . 26
4.13 lcd_backlight . . . 26
4.14 lcd_bl_en . . . 27
4.15 lcd_spi_dc_pin . . . 27
4.16 lcd_gpio_x . . . 27
4.17 lcd_spi_bus_num . . . 27
4.18 lcd_pixel_fmt . . . 27
4.19 fb_buffer_num . . . 28

5 案例说明 29

5.1 menuconfig 配置说明 . . . 29
5.2 源码结构介绍 . . . 29
5.3 应用常用函数说明 . . . 30
5.3.1 bsp_disp_get_screen_width . . . 30
5.3.2 bsp_disp_get_screen_height . . . 30
5.3.3 bsp_disp_lcd_set_layer . . . 30
5.4 测试案例 . . . 31
5.4.1 测试代码路径： . . . 31
5.4.1.1 测试代码说明 . . . 31
5.4.2 测试现象解析 . . . 32

6 FAQ 33

6.1 屏幕适配失败，无显示 . . . 33

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.2 横竖屏旋转相关 33
6.3 多个显示相关 33
6.4 电源配置相关 34
6.5 GPIO 配置说明 34
6.6 怎么判断屏初始化成功 35
6.7 黑屏-无背光 35
6.8 送图无显示 35
6.9 闪屏 35
6.10 画面偏移 36

7 总结 37

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

插　图

图 2-1　3 线模式写时序 5
图 2-2　3 线模式读时序 5
图 2-3　4 线模式写时序 6
图 2-4　4 线模式读时序 6
图 2-5　2 Data Lane 模式写时序 7
图 3-1　创建驱动文件 9
图 3-2　修改 panel 名称 10
图 3-3　修改 sys_config 名称 10
图 3-4　增加 kconfig 10
图 3-5　添加指针 11
图 3-6　添加声明 11
图 3-7　修改 Makefile 11
图 4-1　sys_config 20
图 4-2　rgb565 像素排列 24

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 概述

1.1 文档简介

本文档将介绍 Sunxi 平台 RTOS 上 SPI LCD 屏的调试方法。

1. LCD 调试方法，调试手段。
2. LCD 驱动编写。
3. 板级配置文件中各个属性的解释。
4. 典型 LCD 接口配置。

1.2 目标读者

- SPILCD 驱动开发人员/维护人员
- SPILCD 模块的应用层使用者

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	驱动文件
R128	FreeRTOS	rtos-hal/hal/source/spilcd/
XR875	FreeRTOS	rtos-hal/hal/source/spilcd/
F133	Melis	rtos-hal/hal/source/spilcd/

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.4 文档约定

1.4.1 标志说明

⚠ 注意

- **提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。**

说明

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

1.4.2 地址与数据描述方法约定

本文档在描述地址、数据时遵循如下约定：

表 1-2: 地址与数据描述方法约定

符号	例子	说明
0x	0x0200，0x79	地址或数据以 16 进制表示。
0b	0b010，0b00 000 111	数据采用二进制表示 (寄存器描述除外)。
X	00X，XX1	数据描述中，X 代表 0 或 1。 例如，00X 代表 000 或 001；XX1 代表 001，011，101 或 111。

1.4.3 数值单位约定

本文档在描述数据容量（如 NAND 容量）时，单位词头代表的是 1024 的倍数；描述频率、数据速率等时则代表的是 1000 的倍数。具体如下：

表 1-3: 数值单位约定

类型	符号	对应数值
数据容量（如 NAND 容量）	1 K	1024
	1 M	1 048 576
	1 G	1 073 741 824
频率，数据速率等	1 k	1000

版权所有 © 珠海全志科技股份有限公司。保留一切权利

类型	符号	对应数值
	1 M	1 000 000
	1 G	1 000 000 000

1.5 相关术语介绍

1.5.1 硬件术语

表 1-4: 硬件术语

术语	解释说明
SUNXI	Allwinner 一系列 SoC 硬件平台
LCD	液晶显示屏
SPI	串行外设接口，可以用于传输视频数据，支持 SPI 协议
DBI	显示总线接口，用于传输视频数据，支持 DBI 协议

1.5.2 软件术语

表 1-5: 软件术语

术语	解释说明
HAL	Hardware Abstraction Layer，硬件抽象层
RTOS	Real Time Operatiing System，实时操作系统
GPIO	General Purpose Input/Output，通用输入输出

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块介绍

2.1 基础说明

SPI(Serial Peripheral Interface)LCD 是一种基于 SPI 通信协议的液晶显示器，数据传输特点为通过单一数据线串行传输数据，传输效率较低，适用于小分辨率，引脚资源紧张的场合。

2.2 接口说明

SPI LCD 的 SPI 传输接口不同于嵌入式常使用的 normal SPI，针对 LCD 显示有一套固有的通讯协议。

normal SPI 一般需要 4 根线:

1. CS：片选引脚，一般低电平代表数据传输有效；
2. SCL：时钟引脚；
3. SDO：写数据；
4. SDI：读数据。

SPI LCD 的传输特点主要有：

1. 一般只支持半双工通讯；
2. 主要的传输数据场合，为控制器往 LCD 写数据，读数据场合较少或不需要；
3. 传输的数据需要区分是控制命令 (Command) 还是控制数据 (Parameter) 或显示数据 (Data);

针对特点 1 和特点 2，大多数场合下，只保留一根数据线 SDA 用于传输数据 (也有 LCD 读写引脚分开);

针对特点 3，数据传输时需要额外传输一个数据位，用于指示当前传输的数据，是 Command 还是 Parameter/Data。

这个模式常叫做 3 线模式，如下图所示。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

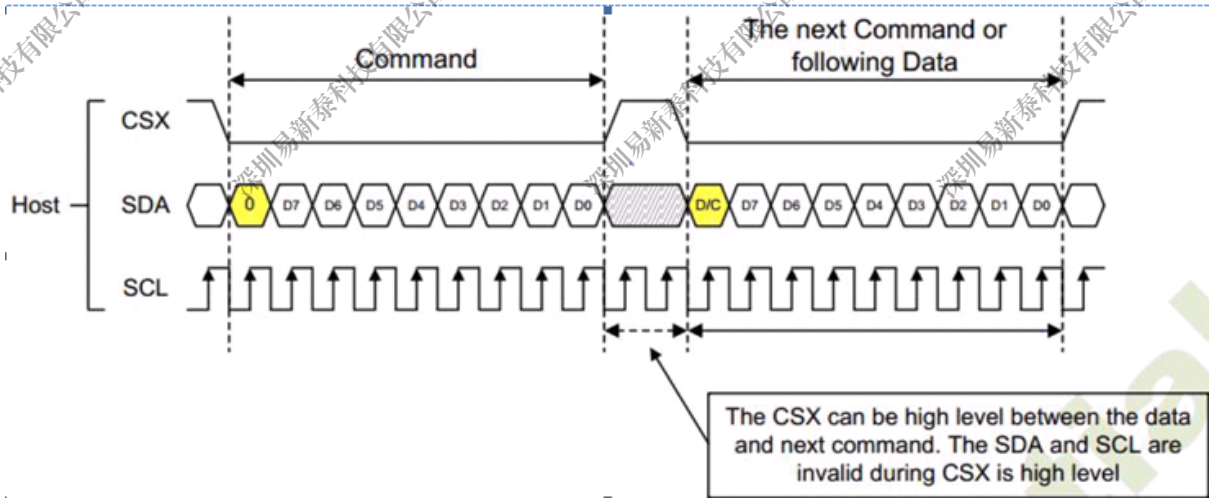


图 2-1: 3 线模式写时序

读数据有两种不同的方式，区别为读写数据线是否独立：

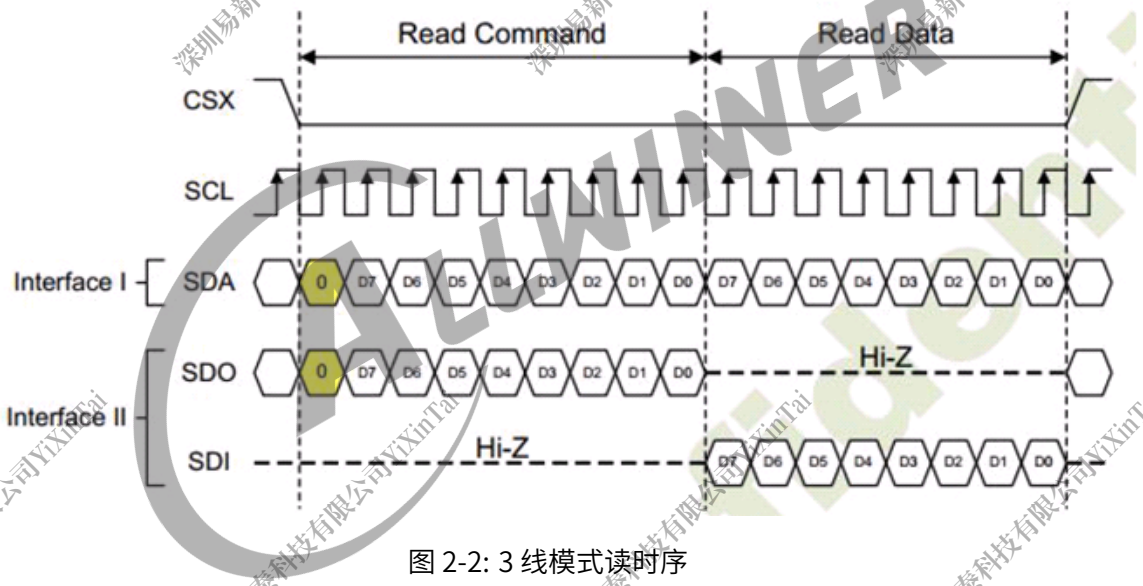


图 2-2: 3 线模式读时序

针对特点 3，也可以额外通过一个 IO 的高低电平，用于区分 Command 还是 Parameter/Data，这根线常称作 **D/C 线**。

这个模式常叫做 4 线模式，如下图所示。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

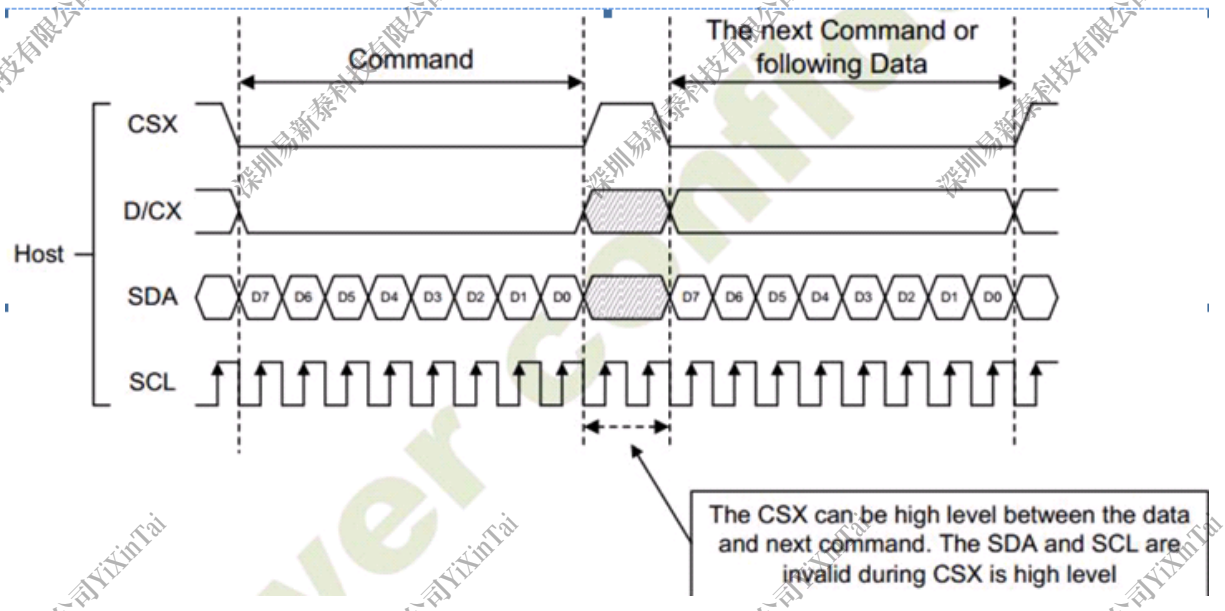


图 2-3: 4 线模式写时序

读数据有两种不同的方式，区别为读写数据线是否独立：

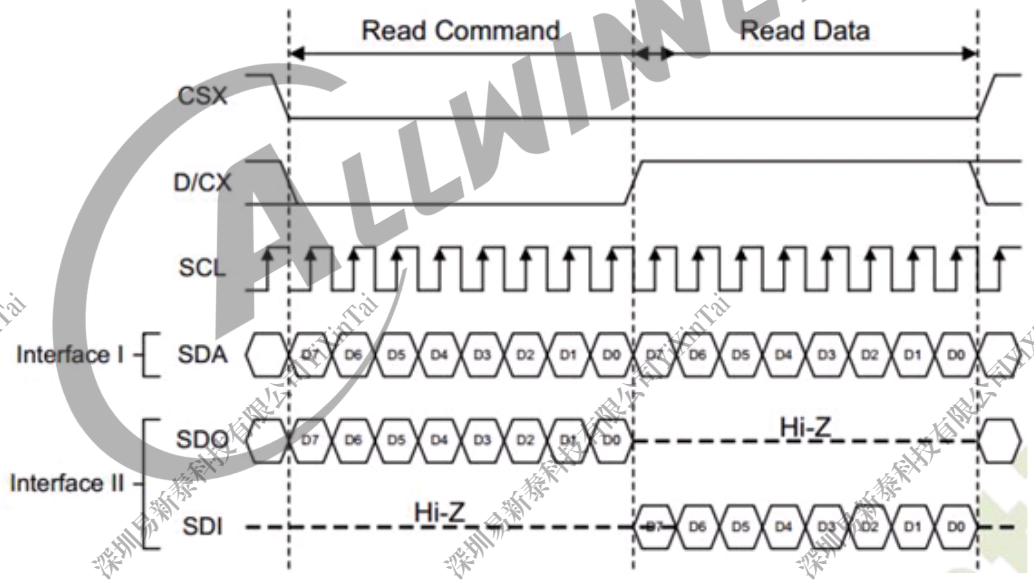


图 2-4: 4 线模式读时序

在 3 线模式或者 4 线模式的基础上，部分 LCD 驱动支持在传输数据时，可以额外使用一根数据线提高传输效率。

这个模式常叫做 2 Data Lane 模式，如下图所示。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

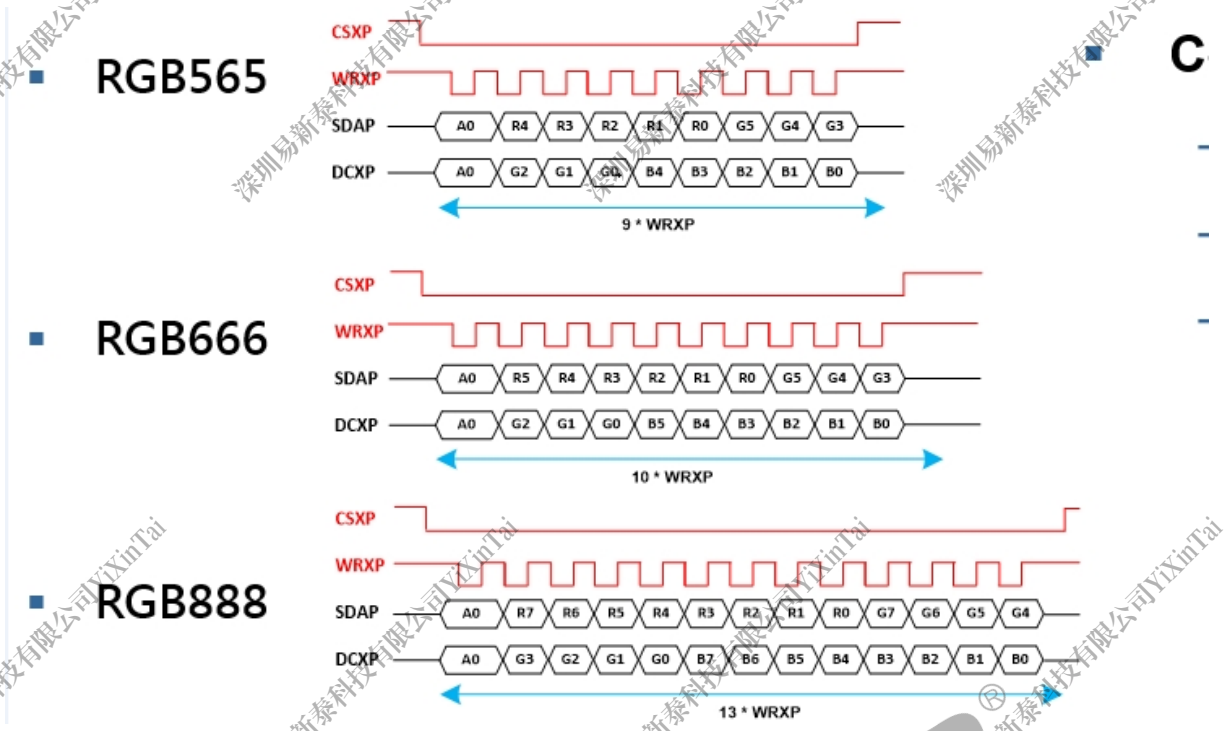


图 2-5: 2 Data Lane 模式写时序

可以看到 normal SPI 只能模拟 4 线模式，针对 3 线模式和 2 Data Lane 模式，部分 AWinner 芯片，把实现这部分显示屏通讯协议的模块接口，称作 **DBI**(Display Bus Interface)，相比 normal SPI，DBI 具有以下特点：

1. 支持 3 线模式和 2 Data Lane 模式，支持 4 线模式的 DC 引脚自动控制；
2. 支持显示数据大小端切换；
3. 支持数据智能输出，可以把数据自动按照 RGB111/RGB444/RGB666/RGB888 的数据形式发送，例如软件数据 RGB888 一个像素点占用 32 位，DBI 可以自动忽略高 8 位或者低 8 位，按照 24 位输出；
4. 支持 TE 触发，部分 LCD 带有 TE 引脚，可以自动控制数据发送。

说明

1. 当前 DBI 模块的 2 Data Lane 模式，仅支持在 3 线模式上增加一根数据线的类型。
2. 更多关于 DBI 的说明请阅读对应芯片的《User_Manual》的 SPI_DBI 部分。

2.3 快速评估

项目开始阶段，需要快速评估项目是否可以使用 SPI LCD 方案。SPI/DBI 的时钟频率是否满足需求，计算公式为：

```
Fhz = Frame_rate * One_pixel_cycle * One_pixel_value
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- Fhz：目标频率；
- Frame_rate：显示帧率；
- One_pixel_cycle：传输一个像素点需要的时钟个数，例如传输一个 RGB565 像素点，3 线模式，需要 17（16 位 + 1 位 D/C 位），4 线需要 16，2 Data Lane 需要 9(16 / 2 + 1)；
- One_pixel_value：一帧像素点个数。

假如 4 线模式需要显示 320 x 480@60fps 的 RGB565 的数据，需要的频率为 147.456Mhz，而 2 Data Lane 模式需要的频率为 82.944Mhz。大部分芯片支持的频率都在 100Mhz 以下，所以使用 SPI 显示大分辨率，高帧率数据并不现实。

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 编写屏驱动

3.1 编写步骤

屏驱动源码位置：

```
rtos-hal/hal/source/spilcd/lcd_fb/panels
```

1. 在屏驱动源码位置下拷贝现有一个屏驱动，包括头文件和源文件，然后将文件名改成有意义的名字，比如屏型号；

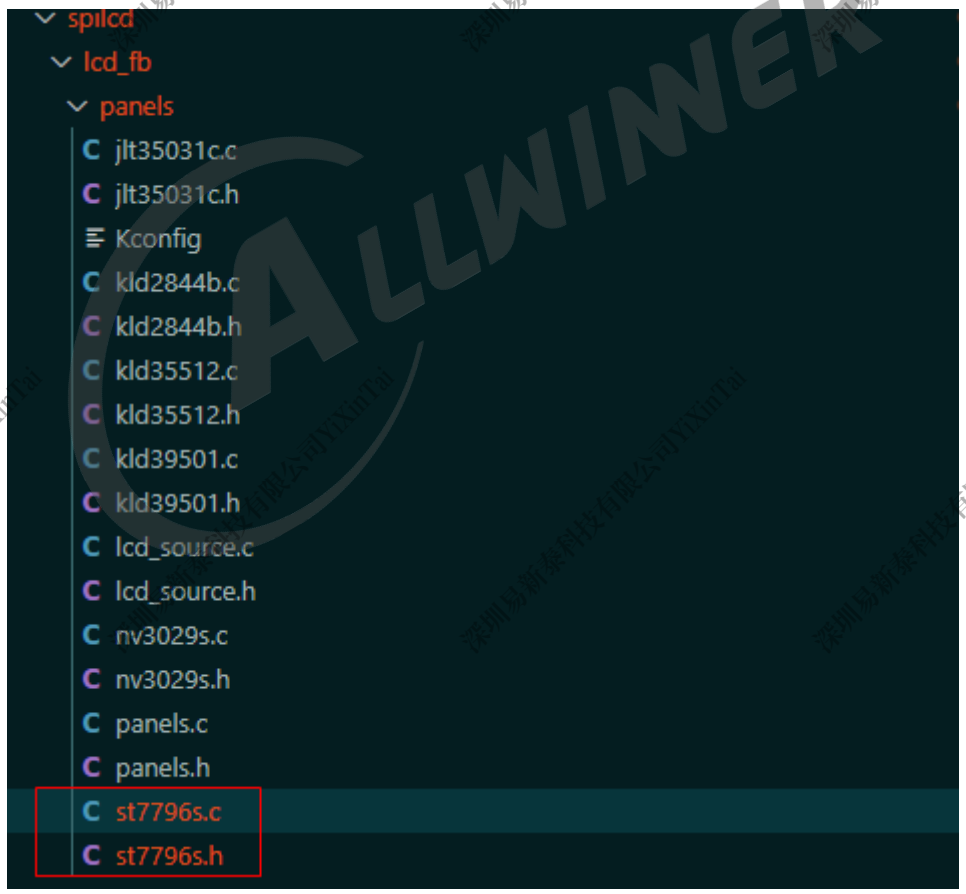


图 3-1: 创建驱动文件

2. 修改源文件中的strcut __lcd_panel变量的名字，以及这个变量成员name的名字，这个名字必须和 sys_config.fex 中 [lcd_fb0]（或板级配置文件中g_lcd0_config）的lcd_driver_name一致；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

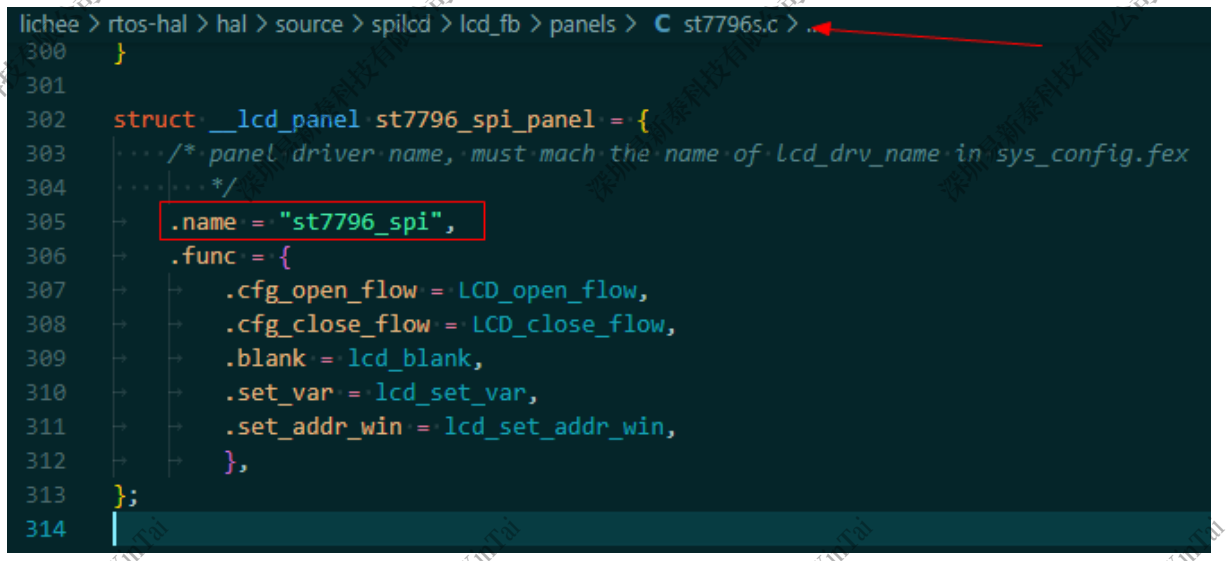


图 3-2: 修改 panel 名称

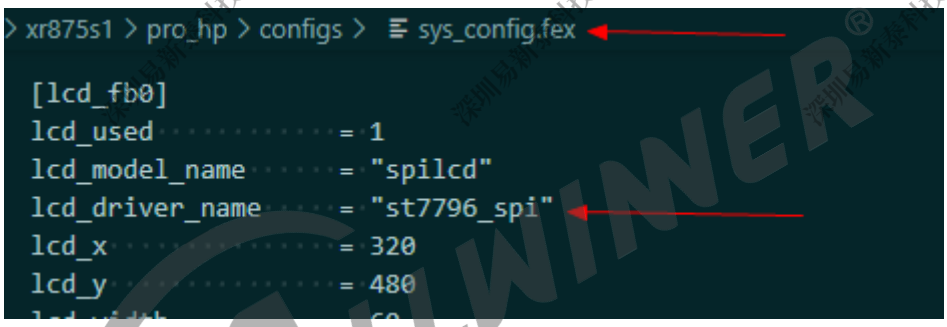


图 3-3: 修改 sys_config 名称

3. 修改rtos-hal/hal/source/spilcd/lcd_fb/panels/Kconfig，新增一个 config，用于在 menuconfig 中配置是否需要使能 LCD 驱动;



图 3-4: 增加 kconfig

4. 在屏驱动目录下修改panel.c和panel.h。在全局结构体变量panel_array中新增刚才添加strcut __lcd_panel 的变量指针。panel.h中新增strcut __lcd_panel的声明。并用宏括起来;

版权所有 © 珠海全志科技股份有限公司。保留一切权利



图 3-5: 添加指针



图 3-6: 添加声明

5. 修改rtos-hal/hal/source/spilcd/lcd_fb/路径下的 Makefile 文件。给 lcd_fb-obj 变量新增刚才加入的源文件对应.o;



图 3-7: 修改 Makefile

6. 实现屏源文件中的LCD_open_flow，LCD_close_flow，LCD_panel_init，LCD_power_on等函数，请看开关屏流程函数解析;

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7. 根据本手册以及屏手册，Driver IC 手册修改 sys_config.fex 中的 [lcd_fb0] 节点 (或显示私有方式的板级配置文件中的g_lcd0_config配置变量) 下面的属性，各个属性的含义请看板级显示配置参数详解。

3.2 开关屏流程函数解析

SPI LCD 屏幕的初始化流程可以简单理解概括如下：

```
main -->

lcd_fb_probe()-->

hal_workqueue_create && hal_work_init : 创建一个线程，用于管理SPI lcd显示；

disp_init_lcd： 从sys_confix.fex中获取显示屏的硬件配置，并初始化LCD的行为函数；

lcd_init： disp_init_lcd中已经获取了 lcd_driver_name ，lcd_init则从panel_array中遍历所有LCD的 name ,两者相匹配，则
        匹配显示屏的cfg_open_flow、cfg_close_flow等操作;

start_process : 启动线程，这时候会去执行lcd_init里面，通过cfg_open_flow注册的函数。

cfg_open_flow，也即是LCD_open_flow，在结构体中赋值：
struct __lcd_panel kld2844b_panel = {
   /* panel driver name, must mach the name of lcd_drv_name in sys_config.fex
     */
   .name = "kld2844b",
   .func = {
      .cfg_open_flow = LCD_open_flow,
      .cfg_close_flow = LCD_close_flow,
      .blank = lcd_blank,
      .set_var = lcd_set_var,
      .set_addr_win = lcd_set_addr_win,
      },
};
```

编写/修改以下代码前，必须先根据 LCD 的数据手册或者屏厂了解清楚 LCD 初始化流程。

关于 LCD_open_flow，也就是开屏流程函数，该函数为注册函数，对应初始化流程的lcd_init，并不会立刻执行, 而是在start_process启动线程后，按顺序执行。

函数常用内容为：

```
static __s32 LCD_open_flow(__u32 sel)
{
   // 打开LCD电源，再延迟10ms。
   LCD_OPEN_FUNC(sel, LCD_power_on,10);

   // 初始化屏，再延迟50ms；(不需要初始化的屏，可省掉此步骤)。
   LCD_OPEN_FUNC(sel, LCD_panel_init, 50);

   // 向屏发送全黑的数据，因为部分显示屏初始化完成后为花屏。
   LCD_OPEN_FUNC(sel, lcd_fb_black_screen, 100);

   // 打开背光
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    LCD_OPEN_FUNC(sel, LCD_bl_open, 0);

    // 可以添加自定义函数
    // LCD_OPEN_FUNC(sel, customer_function, delay);
    return 0;
}
```

建议先跳转到LCD_OPEN_FUNC确认清楚该函数的详细功能。

适配好LCD_power_on，LCD_panel_init，lcd_fb_black_screen三个函数可以正常点屏，下述说明过程中，如果遇到不清楚功能的函数，请查阅常用驱动函数解析。

3.2.1 LCD_power_on

该函数用于控制 LCD 硬件复位，因为有些 LCD 需要满足特定的复位时序，如 SDK 中的kld39501。

```
#define RESET(s, v) sunxi_lcd_gpio_set_value(s, 0, v)

static void LCD_power_on(u32 sel)
{
    /* config lcd_power pin to open lcd power0 */
    sunxi_lcd_power_enable(sel, 0);

    sunxi_lcd_pin_cfg(sel, 1);
    RESET(sel, 1);
    sunxi_lcd_delay_ms(100);
    RESET(sel, 0);
    sunxi_lcd_delay_ms(100);
    RESET(sel, 1);
}
```

3.2.2 LCD_panel_init

这部分代码，主要通过控制命令初始化 LCD，需要严格按照数据手册或屏厂要求或官方例程初始化。一般情况下，我们会以写数据命令结尾，代表后续只需要写显示屏数据即可，不需要再发送控制命令。参考 SDK 中的kld2844b。

```
    sunxi_lcd_cmd_write(sel, 0x11); /* Sleep Out */
    sunxi_lcd_delay_ms(120);
    sunxi_lcd_cmd_write(sel, 0x29); /* Display ON */
    // 2C命令为该显示屏的写内存命令
    sunxi_lcd_cmd_write(sel, 0x2c); /* Display ON */
```

3.2.3 lcd_fb_black_screen

该函数用于发送黑屏数据，也就是发送全 0 数据，lcd_fb_black_screen函数并不需要修改，要修改的是lcd_fb_black_screen调用到的两个函数lcd_set_addr_win和lcd_blank，一个用于设置屏幕开始写数据的位置，一个用于可以支持命令黑屏的 LCD。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
static void address(unsigned int sel, int x, int y, int width, int height)
{
  // 修改为对应的写位置命令
  sunxi_lcd_cmd_write(sel, 0x2a); /* Set coloum address */
  sunxi_lcd_para_write(sel, ((x + lcd_dev_xoffset) >> 8) & 0xff));
  sunxi_lcd_para_write(sel, ((x + lcd_dev_xoffset) & 0xff));
  sunxi_lcd_para_write(sel, ((width + lcd_dev_xoffset >> 8) & 0xff));
  sunxi_lcd_para_write(sel, ((width + lcd_dev_xoffset) & 0xff));
  // 修改为对应的写位置命令
  sunxi_lcd_cmd_write(sel, 0x2b); /* Set row address */
  sunxi_lcd_para_write(sel, ((y + lcd_dev_yoffset >> 8) & 0xff));
  sunxi_lcd_para_write(sel, ((y + lcd_dev_yoffset) & 0xff));
  sunxi_lcd_para_write(sel, ((height + lcd_dev_yoffset >> 8) & 0xff));
  sunxi_lcd_para_write(sel, ((height + lcd_dev_yoffset) & 0xff));
}

static int lcd_set_addr_win(unsigned int sel, int x, int y, int width, int height)
{
  address(sel, x, y, width, height);
  // 一般情况下，只需要修改为显示屏对应的写内存命令即可
  sunxi_lcd_cmd_write(sel, 0x2c); /* Display ON */
  return 0;
}

static int lcd_blank(unsigned int sel, unsigned int en)
{
  // 部分显示屏支持黑屏命令，不支持的情况下，该函数可以直接返回0
  if (en)
    sunxi_lcd_cmd_write(sel, 0x28);
  else
    sunxi_lcd_cmd_write(sel, 0x29);

  sunxi_lcd_cmd_write(sel, 0x2c); /* Display ON */
  return 0;
}
```

3.3 常用驱动函数解析

3.3.1 LCD_OPEN_FUNC

- 功能：注册开屏步骤函数到开屏流程中，记住这里是注册不是执行！
- 原型：void LCD_OPEN_FUNC(__u32 sel, LCD_FUNC func, __u32 delay)；
- 参数：
  - sel：显示屏序号，如配置文件中定义 [lcd_fb0]，则 sel 为 0 则表示选择该 LCD；
  - func：注册自定义函数，形式统一为void user_defined_func(__u32 sel)；
  - delay：执行完 func 后的延时长度，单位 ms，注意这里的数值请按照屏手册规定去填；
- 返回：无返回。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.3.2 sunxi_lcd_delay_ms

- 功能：延时函数，实现毫秒级别的延时。
- 原型：s32 sunxi_lcd_delay_ms(u32 ms);
- 参数：
  - ms：延时时间，单位为 ms;
- 返回：固定返回 0。
- 备注：RTOS 不支持 us 级延时。

3.3.3 sunxi_lcd_backlight_enable

- 功能：打开背光，操作的是配置文件中定义的lcd_bl_en，如果没有配置，该 API 无效。
- 原型：void sunxi_lcd_backlight_enable(u32 screen_id);
- 参数：
  - screen_id: 显示屏序号，如配置文件中定义 [lcd_fb0]，则 screen_id 为 0 则表示选择该 LCD;
- 返回：
  - 0：执行成功;
  - other：执行失败。

3.3.4 sunxi_lcd_backlight_disable

- 功能：关闭背光，操作的是配置文件中定义的lcd_bl_en，如果没有配置，该 API 无效。
- 原型：void sunxi_lcd_backlight_disable(u32 screen_id);
- 参数：
  - screen_id: 显示屏序号，如配置文件中定义 [lcd_fb0]，则 screen_id 为 0 则表示选择该 LCD;
- 返回：
  - 0：执行成功;
  - other：执行失败。

3.3.5 sunxi_lcd_pwm_enable

- 功能：打开 pwm 控制器，将往外输出 pwm 波形。操作的是配置文件中定义的lcd_pwm_ch, 如果没有配置，该 API 无效。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 原型：s32 sunxi_lcd_pwm_enable(u32 screen_id);
- 参数：
  - screen_id: 显示屏序号，如配置文件中定义 [lcd_fb0]，则 screen_id 为 0 则表示选择该 LCD；
- 返回：
  - 0：执行成功；
  - other：执行失败。

3.3.6 sunxi_lcd_pwm_disable

- 功能：关闭 pwm 控制器，停止 PWM 输出。操作的是配置文件中定义的lcd_pwm_ch, 如果没有配置，该 API 无效。
- 原型：s32 sunxi_lcd_pwm_disable(u32 screen_id);
- 参数：
  - screen_id: 显示屏序号，如配置文件中定义 [lcd_fb0]，则 screen_id 为 0 则表示选择该 LCD；
- 返回：
  - 0：执行成功；
  - other：执行失败。

3.3.7 sunxi_lcd_power_enable

- 功能：打开 Lcd 电源，操作的是配置文件中定义的 lcd_power/lcd_power1/lcd_power2。
- 原型：void sunxi_lcd_power_enable(u32 screen_id, u32 pwr_id);
- 参数：
  - screen_id: 显示屏序号，如配置文件中定义 [lcd_fb0]，则 screen_id 为 0 则表示选择该 LCD；
  - pwr_id：
    - pwr_id = 0：对应于配置文件中的 lcd_power。
    - pwr_id = 1：对应于配置文件中的 lcd_power1。
    - pwr_id = 2：对应于配置文件中的 lcd_power2。
    - pwr_id = 3：对应于配置文件中的 lcd_power3。
- 返回：
  - 0：执行成功；
  - other：执行失败。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.3.8 sunxi_lcd_power_disable

- 功能：关闭 Lcd 电源，操作的是配置文件中定义的 lcd_power/lcd_power1/lcd_power2。
- 原型：void sunxi_lcd_power_disable(u32 screen_id, u32 pwr_id);
- 参数：
  - 同sunxi_lcd_power_enable；
- 返回：
  - 0： 执行成功；
  - other：执行失败。

3.3.9 sunxi_lcd_cmd_write

- 功能：发送命令。
- 原型：s32 sunxi_lcd_cmd_write(u32 screen_id, u8 cmd);
- 参数：
  - screen_id: 显示屏序号，如配置文件中定义 [lcd_fb0]，则 screen_id 为 0 则表示选择该 LCD；
  - cmd: 需要向 LCD 发送的命令；
- 返回：
  - 0： 执行成功；
  - other：执行失败。

3.3.10 sunxi_lcd_para_write

- 功能：发送参数。
- 原型：s32 sunxi_lcd_para_write(u32 screen_id, u8 para);
- 参数：
  - screen_id: 显示屏序号，如配置文件中定义 [lcd_fb0]，则 screen_id 为 0 则表示选择该 LCD；
  - para: 需要向 LCD 发送的数据；
- 返回：
  - 0： 执行成功；
  - other：执行失败。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.3.11 sunxi_lcd_gpio_set_value

- 功能：LCD_GPIO PIN 脚上输出高电平或低电平。
- 原型：s32 sunxi_lcd_gpio_set_value(u32 screen_id, u32 io_index, u32 value);
- 参数：
  - screen_id: 显示屏序号，如配置文件中定义 [lcd_fb0]，则 screen_id 为 0 则表示选择该 LCD；
  - io_index:
    - io_index = 0：对应于配置文件中的 lcd_gpio_0。
    - io_index = 1：对应于配置文件中的 lcd_gpio_1。
    - io_index = 2：对应于配置文件中的 lcd_gpio_2。
    - io_index = 3：对应于配置文件中的 lcd_gpio_3。
  - value:
    - value = 0：对应 IO 输出低电平。
    - Value = 1：对应 IO 输出高电平。
- 返回：
  - 0：执行成功；
  - other：执行失败。
- 备注：只用于该 GPIO 定义为输出的情形。

3.3.12 sunxi_lcd_gpio_set_direction

- 功能：设置 LCD_GPIO PIN 脚为输入或输出模式。
- 原型：s32 sunxi_lcd_gpio_set_direction(u32 screen_id, u32 io_index, u32 direction);
- 参数：
  - screen_id: 显示屏序号，如配置文件中定义 [lcd_fb0]，则 screen_id 为 0 则表示选择该 LCD；
  - io_index:
    - io_index = 0：对应于配置文件中的 lcd_gpio_0。
    - io_index = 1：对应于配置文件中的 lcd_gpio_1。
    - io_index = 2：对应于配置文件中的 lcd_gpio_2。
    - io_index = 3：对应于配置文件中的 lcd_gpio_3。
  - direction:
    - direction = 0：对应 IO 设置为输入。
    - direction = 1：对应 IO 设置为输出。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 模块配置介绍

如前文介绍，SPI LCD 驱动中，软件需要配置正确的配置才能成功初始化，适配显示屏时，需要逐一确认配置。

4.1 板级显示配置说明

当前板级显示支持两种配置方法，一是使用 sys_config.fex 的方式进行配置，二是在不支持 sys_config.fex 情况下，可以通过显示驱动私有的方式进行配置。下面分别对两种方式进行说明。

4.1.1 使用 sys_config.fex 的方式进行配置

Melis 系统路径：source/project/芯片-方案名/configs/sys_config.fex

FreeRTOS 系统路径：board/芯片名/方案名/configs/sys_config.fex

说明

配置文件具体要看芯片方案所实际使用的，也可能使用的配置文件名称为 sys_config_xxx.fex(xx 是存储方案的标识，例如 sys_config_nor.cfg、sys_config_nand.cfg)。

具体配置举例如下：

深圳易新泰科技有限公司YiXinTai

```
[lcd_fb0]
lcd_used            = 1
lcd_model_name      = "spilcd"
lcd_driver_name     = "kld2844b"
lcd_x               = 240
lcd_y               = 320
lcd_width           = 60
lcd_height          = 95
lcd_data_speed      = 60
lcd_pwm_used        = 1
lcd_pwm_ch          = 6
lcd_pwm_freq        = 5000
lcd_pwm_pol         = 0
lcd_if              = 1
lcd_pixel_fmt       = 0
lcd_dbi_fmt         = 3
lcd_dbi_clk_mode    = 0
lcd_dbi_te          = 1
fb_buffer_num       = 2
lcd_dbi_if          = 4
lcd_rgb_order       = 0
lcd_fps             = 60
lcd_spi_bus_num     = 1
lcd_frm             = 1
lcd_gamma_en        = 1
lcd_backlight       = 100

lcd_power_num       = 0
lcd_gpio_regu_num   = 0
lcd_bl_percent_num  = 0

;lcd_bl_en           = port:PB04<1><0><default><1>
lcd_spi_dc_pin      = port:PA19<1><0><3><1>
```

图 4-1: sys_config

如图中所示，节点 lcd_fb0 内为 lcd 相应部分的配置，而对于 lcd_fb0 可以配置的属性以及各属性的可取值详见板级显示配置参数详解。

4.1.2 使用显示私有方式进行配置

路径：rtos-hal/hal/source/spilcd/soc/

具体板级显示配置文件可参考该路径下的 kld2844b_config.c 配置文件。

该文件模仿 sys_config.fex 来配置一些板级相关的资源，源文件中需要定义 4 个全局变量：g_lcd0_config、g_lcd1_config、g_lcd0_config_len 和 g_lcd1_config_len，且变量名字固定，不能修改。

具体说明如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1. g_lcd0_config：第一个屏的配置变量，struct property_t 数据类型。
2. g_lcd1_config：第二个屏的配置变量，struct property_t 数据类型。
3. g_lcd0_config_len 和 g_lcd1_config_len 是对应上面两个数组变量的长度，照搬即可。

struct property_t 数据类型，用于定义一个属性的信息：

1. 属性名字。对应其成员 name，字符串。
2. 属性类型。对应其成员 type，看 enum proerty_type 的定义，共有整型，字符串，GPIO，专用 pin 和电源。
3. 属性值。根据上面的属性类型来选择 union 中成员来赋值。

对于上述常用的属性类型举例如下：

整型：

```
{
    .name = "lcd_used",
    .type = PROPERTY_INTGER,
    .v.value = 1,
},
```

字符串：

```
{
    .name = "lcd_driver_name",
    .type = PROPERTY_STRING,
    .v.str = "kld2844b",
},
```

GPIO：

```
{
    .name = "lcd_spi_dc_pin",
    .type = PROPERTY_GPIO,
    .v.gpio_list = {
        .gpio = GPIOA(19),
        .mul_sel = GPIO_DIRECTION_OUTPUT,
        .pull = 0,
        .drv_level = 3,
        .data = 1,
    },
},
```

g_lcd0_config 的含义与前述的使用 sys_config.fex 方式中的 lcd_fb0 一致。对于 g_lcd0_config 可以配置的属性以及各属性的可取值详见板级显示配置参数详解。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2 板级显示配置参数详解

4.2.1 lcd_driver_name

- 功能：Lcd 屏驱动的名字 (字符串)；
- 参数：
  - 必须与屏驱动中strcut __lcd_panel变量的name成员一致。

4.2.2 lcd_model_name

- 功能：Lcd 屏模型名字，非必须，可以用于同个屏驱动中进一步区分不同屏；
- 参数：
  - 自定义，如spilcd.

4.2.3 lcd_if

- 功能：设置接口类型；
- 参数：
  - 0：spi 接口；
  - 1：dbi 接口。
- 备注：如果设置了 dbi 接口，那么还需要进一步区分 dbi 接口，需要设置lcd_dbi_if。

4.2.4 lcd_dbi_if

- 功能：设置 DBI 接口类型；
- 参数：
  - 0：L3I1，3 线模式，读写同一根数据线；
  - 1：L3I2，3 线模式，读写不同数据线；
  - 2：L4I1，4 线模式，读写同一根数据线；
  - 3：L4I2，4 线模式，读写不同数据线；
  - 4：D2LI，2 Data Lane 模式，读数据同 L3I1；
- 备注：这个参数只有在 lcd_if=1 时才有效。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2.5 lcd_dbi_fmt

- 功能：DBI 接口像素格式。
- 参数：
  - 0：RGB111
  - 1：RGB444
  - 2：RGB565
  - 3：RGB666
  - 4：RGB888
- 备注：
  - 选择的依据是接收端屏 Driver IC 的支持情况，请查看 Driver IC 手册或询问屏厂。
  - 必须配合lcd_pixel_fmt的选择，比如说选 RGB565 时，lcd_pixel_fmt 也要选 565 格式。

4.2.6 lcd_dbi_te

- 功能：使能 te 触发。
- 参数：
  - 0: 禁止 te
  - 1: 下降沿触发
  - 2: 上升沿触发
- 备注：
  - te 即 (Tearing effect)，也就是撕裂的意思，由于读写不同导致撕裂现象，te 脚的功能就是用于同步读写，te 脚的频率也就是屏的刷新率，所以 te 脚也可以看做 vsync 脚 (垂直同步脚)；
  - 这个参数只有在 lcd_if=1 时才有效；
  - 需要同步配置屏幕的 te 相关寄存器，如使能 te 和配置 te 频率；
  - 硬件上，屏的 te 引脚和 IC 的 te 引脚记得互相连接。

4.2.7 lcd_dbi_clk_mode

- 功能：选择 dbi 时钟的行为模式。
- 参数：
  - 0：一直保持，无论发不发数据都有时钟；
  - 1：自动停止。有数据就有时钟，没发数据就没有；
- 备注：
  - 使用逻辑分析仪抓数据时，建议设置为 1。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.3 lcd_rgb_order

- 功能：输入图像数据 rgb 顺序识别设置。
- 参数：
  - 0:RGB；
  - 1:RBG；
  - 2:GRB；
  - 3:GBR；
  - 4:BRG；
  - 5:BGR；
  - 6:G_1RBG_0；
  - 7:G_0RBG_1；
  - 8:G_1BRG_0；
  - 9:G_0BRG_1；
- 备注：
  - 仅当 lcd_if=1 时有效；
  - 输出模式为 RGB565 时，1 ~ 4 无效；非 RGB565 时，6 ~ 9 无效；

4.3.1 针对 rgb565 格式额外说明

rgb565 格式会遇到大小端问题，arm 平台和 PC 平台存储都是小端 (little endian)，低字节放在低地址，高字节放在高地址），但是许多 spi 屏都是默认大端 (Big Endian)。

也就是存储的字节顺序和发送的字节顺序不对应。

这个时候选择 0 或 5，DBI 接口就会自动将小端转成大端。

如果遇到默认是小端的 spi 屏，则需要选择 6~9，DBI 接口会自动用回小端方式。

6 以上格式这样解释：

R 是 5 比特，G 是 6 比特，B 是 5 比特，再把 G 拆成高 3 位 (G_1) 和低 3 位 (G_0) 所以以下两种顺序：

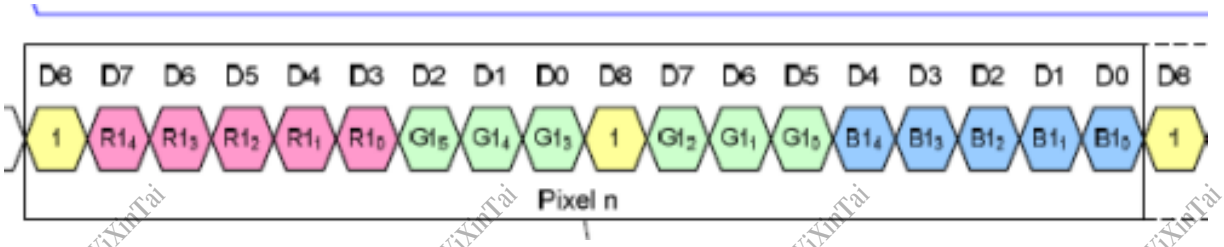


图 4-2: rgb565 像素排列

深圳易新泰科技有限公司YiXinTai

1. R-G_1-G_0-B，大端。
2. G_0-B-R-G_1，对应上面的 9，小端。

4.4 lcd_x

- 功能：显示屏的水平像素数量配置。
- 参数：根据显示屏尺寸配置。
- 备注：如果屏支持横竖旋转，那么 lcd_x 和 lcd_y 也要对调。

4.5 lcd_y

- 功能：显示屏的行数配置。
- 参数：根据显示屏尺寸配置。
- 备注：注意如果屏支持横竖旋转，那么 lcd_x 和 lcd_y 也要对调。

4.6 lcd_data_speed

- 功能：用于设置 spi/dbi 接口时钟的频率，单位 MHz。
- 参数：目标波特率。
- 备注：
  - 最大限制请阅读对应芯片的 user manual；
  - 如果无法配置，软件会自动配置临近的波特率；
  - 需要和显示屏最大支持频率相匹配。

4.7 lcd_fps

- 功能：设置屏的刷新率，单位 Hz。
- 参数：目标刷新率。
- 备注：当lcd_dbi_te使能时，这个值设置无效。

4.8 lcd_pwm_used

- 功能：配置是否使用 pwm。
- 参数：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

  - 0：不使用 PWM 控制背光；
  - 1：使能 PWM 控制背光。

4.9 lcd_pwm_ch

- 功能：配置 PWM 使用的通道。
- 参数：目标通道数。
- 备注：
  - 对应 IO 口需要配置成 PWM 功能使能；
  - lcd_pwm_used 为 0 时，该配置无效。

4.10 lcd_pwm_freq

- 功能：配置 PWM 信号的频率，单位为 Hz。
- 参数：目标频率。

4.11 lcd_pwm_pol

- 功能：配置 PWM 信号的占空比的极性。
- 参数：
  - 0：active high；
  - 1：active low。

4.12 lcd_pwm_max_limit

- 功能：背光最高限制，以亮度值表示。
- 参数：0~255.
- 备注：比如 150，则表示背光最高只能调到 150，0～255 范围内的亮度值将会被线性映射到 0～150 范围内。用于控制最高背光亮度，节省功耗。

4.13 lcd_backlight

- 功能：PWM 默认背光值。
- 参数：0~255，值越大越亮。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.14 lcd_bl_en

- 功能：背光使能脚定义。
- 参数：
  - 举例说明：lcd_bl_en = port:PA19<1><0><3><1>
- 备注：需要硬件支持。

4.15 lcd_spi_dc_pin

- 功能：指定作为 D/C(区分 Command 或者 Data) 的管脚。
- 参数：
  - 举例说明：lcd_spi_dc_pin = port:PA19<1><0><3><1>
- 备注：仅适用 lcd_if=0，也就是 SPI 接口的场合。

4.16 lcd_gpio_x

- 功能：自定义引脚配置。
- 参数：
  - 举例说明：lcd_gpio_0 = port:PA19<1><0><3><1>
- 备注：
  - 可以使用多个自定义引脚，如 lcd_gpio_0，lcd_gpio_1 等；
  - 软件上配合sunxi_lcd_gpio_set_value使用。

4.17 lcd_spi_bus_num

- 功能：选择 spi 总线 id。
- 参数：1.
- 备注：只有 spi1 支持 DBI 协议，所以这里只能选择 1。

4.18 lcd_pixel_fmt

- 功能：选择传输数据的像素格式。
- 参数：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

  - 0：LCDFB_FORMAT_ARGB_8888；
  - 1：LCDFB_FORMAT_ABGR_8888；
  - 2：LCDFB_FORMAT_RGBA_8888；
  - 3：LCDFB_FORMAT_BGRA_8888；
  - 4：LCDFB_FORMAT_XRGB_8888；
  - 5：LCDFB_FORMAT_XBGR_8888；
  - 6：LCDFB_FORMAT_RGBX_8888；
  - 7：LCDFB_FORMAT_BGRX_8888；
  - 8：LCDFB_FORMAT_RGB_888；
  - 9：LCDFB_FORMAT_BGR_888；
  - 10：LCDFB_FORMAT_RGB_565；
  - 11：LCDFB_FORMAT_BGR_565；
  - 12：LCDFB_FORMAT_ARGB_4444；
  - 13：LCDFB_FORMAT_ABGR_4444；
  - 14：LCDFB_FORMAT_RGBA_4444；
  - 15：LCDFB_FORMAT_BGRA_4444；
  - 16：LCDFB_FORMAT_ABGR_1555；
  - 17：LCDFB_FORMAT_ABGR_1555；
  - 18：LCDFB_FORMAT_RGBA_5551；
  - 19：LCDFB_FORMAT_BGRA_5551；
- 备注：
  - DBI 接口只支持 RGB32 和 RGB16 的情况，SPI 接口只支持 RGB16 的情况；
  - 需要和lcd_rgb_order相适配。

4.19 fb_buffer_num

- 功能：配置显示 framebuffer 数量。
- 参数：1 以上。
- 备注：为了平滑显示，这里一般是 2 个，为了省内存也可以改成 1。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 案例说明

5.1 menuconfig 配置说明

SPILCD 模块 menuconfig 的配置如下 (以选择 kld2844b 屏为例)：

```
Drivers Options  --->
  soc related device drivers  --->
    SPILCD Devices  --->
      [*] DISP Driver Support(spi_lcd)
      [*]   spilcd hal APIs test        //spilcd模块测试用例
      LCD_FB panels select  --->        //spilcd屏驱动配置
        [*] LCD support kld2844B panel
        [ ] LCD support kld35512 panel
      Board select  --->
        [ ] board kld2844b support     //板级显示使用显示驱动私有方式的配置项，而使用sys_config.fex方式不用配置
```

另外可能需依赖的配置项有：

- 驱动相关
  - DRIVERS_SPI
  - DRIVERS_DBI
  - DRIVERS_PWM
- 测试相关
  - CONFIG_HAL_TEST_SPILCD

5.2 源码结构介绍

源码结构及主要驱动文件如下：

```
spilcd/
├── lcd_fb/
│   ├── dev_lcd_fb.c          # spilcd driver 层
│   ├── disp_display.c
│   ├── disp_lcd.c
│   ├── lcd_fb_intf.c
│   └── panels/               # lcd驱动相关
│       ├── kld2844b.c
│       ├── lcd_source.c
│       ├── panels.c
│       └── panels.h
└── soc/
```

深圳易新泰科技有限公司YiXinTai

```
└── disp_board_config.c        # 板级配置解析
└── kld2844b_config.c          # 显示私有方式的板级配置文件
```

5.3 应用常用函数说明

5.3.1 bsp_disp_get_screen_width

- 功能：获取显示屏的宽。
- 原型：s32 bsp_disp_get_screen_width(u32 disp)。
- 参数：
  - disp：显示屏序号，如配置文件中定义 [lcd_fb0]，则 disp 为 0 则表示选择该 LCD；
- 返回：
  - 对应lcd_x的数值。

5.3.2 bsp_disp_get_screen_height

- 功能：获取显示屏的宽。
- 原型：s32 bsp_disp_get_screen_height(u32 disp)。
- 参数：
  - disp：显示屏序号，如配置文件中定义 [lcd_fb0]，则 disp 为 0 则表示选择该 LCD；
- 返回：
  - 对应lcd_y的数值。

5.3.3 bsp_disp_lcd_set_layer

- 功能：输出显示数据。
- 原型：s32 bsp_disp_lcd_set_layer(unsigned int disp, struct fb_info *p_info)。
- 参数：
  - disp：显示屏序号，如配置文件中定义 [lcd_fb0]，则 disp 为 0 则表示选择该 LCD；
  - p_info：显示配置。
- 返回：
  - 0：显示成功。
  - 1：参数异常。
- 备注：针对 p_info 数值的初始化，可以参考下述代码。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
static void spi_fb_init(void *buff_addr, unsigned width, unsigned height)
{
    // 显示数据的首地址
    g_fb_info.screen_base = buff_addr;
    // 配置显示数据的大小，显示整个屏幕则设置为整个屏幕大小
    g_fb_info.var.xres = width;
    g_fb_info.var.yres = height;
    // 配置开始显示的位置，显示整个屏幕则设置为0，0
    g_fb_info.var.xoffset = 0;
    g_fb_info.var.yoffset = 0;
    // 显示格式，同lcd_pixel_fmt配置，可以不设
    g_fb_info.var.lcd_pixel_fmt = LCDFB_FORMAT_RGB_565;
    // 显示一行的数据长度，输出格式为rgb565，则DISPLAY_BPP为2
    // 如果输出格式为rgb888，则DISPLAY_BPP为4
    // 可不设
    g_fb_info.fix.line_length = (width * DISPLAY_BPP);
}
```

5.4 测试案例

5.4.1 测试代码路径：

```
rtos-hal/hal/test/spilcd/test_spilcd.c
```

5.4.1.1 测试代码说明

```
static void lcdfb_fb_init(uint32_t yoffset, struct fb_info *p_info)
{
    p_info->screen_base = fbsmem_start;
    p_info->var.xres = width;
    p_info->var.yres = height;
    p_info->var.xoffset = 0;
    p_info->var.yoffset = yoffset;
}

int show_rgb(unsigned int sel)
{
    int i = 0, ret = -1;
    struct fb_info fb_info;
    int bpp = 4;
    unsigned char color[4] = {0xff,0x0,0xff,0x0};

    /* 获取屏幕尺寸，准备显示整个屏幕 */
    width = bsp_disp_get_screen_width(sel);
    height = bsp_disp_get_screen_height(sel);
    screensize = width * bpp * height;

    /* 大批量数据需要DMA发送，申请的数据需要按照cacheline对齐 */
    fbsmem_start = hal_malloc_coherent(screensize);

    hal_log_info("width = %d, height = %d, screensize = %d, fbsmem_start = %x\n",
        width, height, screensize, fbsmem_start);
```

深圳易新泰科技有限公司YiXinTai

```
    memset(fbsmem_start, 0, screensize);
    /* 所有显示数据按照0xff 0 0xff 0...的方式保存 */
    for (i = 0; i < screensize / bpp; ++i) {
        memcpy(fbsmem_start+i*bpp, color, bpp);
    }

    memset(&fb_info, 0, sizeof(struct fb_info));
    lcdfb_fb_init(0, &fb_info);

    /* 发送前需要请cache，保证数据已经写入内存 */
    hal_dcache_clean((unsigned long)fbsmem_start, screensize);

    /* 推送数据到显示屏 */
    bsp_disp_lcd_set_layer(sel, &fb_info);

    hal_free_coherent(fbsmem_start);
    return ret;
}
```

5.4.2 测试现象解析

输入命令 test_spilcd 后，可能根据配置的不同，屏幕显示不同的颜色。

1. DBI 接口，配置成 RGB888 通过 DBI 发送出去的数据，每个像素数据为：0xff，0x00，0xff，R 和 B 分量为 ff，G 分量为 0，显示为**紫色**。
2. DBI 接口，配置成 RGB565 通过 DBI 发送出去的数据，自动做了大小端适配，所以每个像素点的数据为 0x00，0xff，得到 R 分量为 0，G 分量为 28，B 分量为 255，显示为**蓝色**。
3. SPI 接口，配置成 RGB565 通过 SPI 发送出去的数据，SPI 模块不会做大小端适配，所以每个像素点的数据为 0xff，0x00，得到 R 分量为 255，R 分量为 28，B 分量为 0，显示为**黄色**。部分 SPI LCD 可以通过修改寄存器的方式自行切换大小端，如 SDK 已经适配的 kld2844b 的 B0 寄存器，因为大小端自动切换，所以即使使用 SPI 接口，依旧显示**蓝色**。如果 LCD 不具备切换大小端的能力，需要软件把显示数据切换大小端。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 FAQ

6.1 屏幕适配失败，无显示

1. 先准备好逻辑分析仪，核实是否有数据正常输出。
2. 如果没有数据发送，需要进行以下排查：
   - 排查配置文件是否有 IO 口配置冲突。
   - 在显示屏驱动代码的LCD_panel_init里面添加 log，确保代码正确进入到了初始化函数，如果没有进入，参照编写屏驱动重新检查是否按要求执行。
   - 显示屏是否带 TE 引脚？如果带 TE 引脚，检查 TE 引脚是否有输出，或者没 TE 引脚，但是lcd_dbi_te配置为了 1。
   - 单独运行一次 SPI/DBI 的 test demo，确保模块功能是正常的，或者硬件连线是正确的。
3. 如果有数据发送，需要进行以下排查：
   - 优先检查数据发送是否正确，如果不正确，也需要排查是否有 IO 口配置冲突。
   - 如果发送的命令是正确的，但是发送的显示数据不正确，排查lcd_dbi_fmt，lcd_pixel_fmt，lcd_rgb_order等配置是否正确。
   - 如果都正确，把逻辑分析仪数据发给屏厂核实初始化时序是否正确，或者对照能够成功初始化的板子的时序寻找差异。

6.2 横竖屏旋转相关

1. 平台没有硬件旋转功能，软件旋转太慢而且耗费 CPU。
2. 不少 spi 屏支持内部旋转，需要在屏驱动初始化的时候进行设置。
3. lcd_fb0 的配置中需要将lcd_x和lcd_y的值对调，此时软件将屏视为横屏。
4. 屏内部旋转出图效果可能会变差，建议选屏的时候直接选好方向。

6.3 多个显示相关

1. 确定硬件有没有多余的 spi/dbi 接口。
2. 需要在 sys_config.fex 里面新增 lcd_fb1，配置方式与 lcd_fb0 一样 (或者在显示私有方式的板级配置文件里面新增g_lcd1_config，配置方式与 g_lcd0_config 一样)，其中lcd_spi_bus_num不能一样。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.4 电源配置相关

1. 有多个电源的话，就用lcd_power1，lcd_power2...lcd_power<n>，然后屏驱动里面调用sunxi_lcd_power_enable接口即可。
2. 可以使用 lcd_gpio_0 自行实现电源控制逻辑。

6.5 GPIO 配置说明

lcd_bl_en、lcd_spi_dc_pin以及lcd_gpio_x都是属于 GPIO 属性类型。

下面以 lcd_spi_dc_pin 为例，具体说明板级显示配置的 sys_config.fex 方式和显示私有方式的 GPIO 属性的参数值含义：

1. 板级显示配置的 sys_config.fex 方式

```
lcd_spi_dc_pin  = port:PA19<1><0><3><1>
```

引脚说明：port: 端口 < 复用功能 >< 上下拉 >< 驱动能力 >< 输出值 >

等式右边的从左到右 5 个字段代表的具体含义如下：

- PA19：端口，表示 GPIO 管脚。PA表示 PA 组管脚，19表示第 19 根管脚；即 PA19 管脚。
- <1>：复用功能，表示 GPIO 复用类型。1 表示将 PA19 选择为通用 GPIO 功能，0 为输入，1 为输出。
- <0>：上下拉，表示内置电阻。使用 0 的话，表示内部电阻高阻态，如果是 1 则是内部电阻上拉，2 就代表内部电阻下拉。其它数据无效。
- <3>：表示驱动能力。1 是默认等级，数字越大驱动能力越强，最大是 3。
- <1>：表示默认输出电平值。0 为低电平，1 为高电平。

2. 板级显示配置的显示私有方式

```
{
    .name = "lcd_spi_dc_pin",
    .type = PROPERTY_GPIO,
    .v.gpio_list = {
        .gpio = GPIOA(19),
        .mul_sel = GPIO_DIRECTION_OUTPUT,
        .pull = 0,
        .drv_level = 3,
        .data = 1,
    },
},
```

gpio_list 内的成员变量及其具体含义如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- gpio：表示 GPIO 管脚。GPIOA表示 PA 组管脚，(19)表示第 19 根管脚；即 PA19 管脚。
- mul_sel：表示 GPIO 复用类型。
- pull：表示内置电阻。使用 0 的话，表示内部电阻高阻态，如果是 1 则是内部电阻上拉，2 就代表内部电阻下拉。其它数据无效。
- drv_level：表示驱动能力。1 是默认等级，数字越大驱动能力越强，最大是 3。
- data：表示默认输出电平值。0 为低电平，1 为高电平。

6.6 怎么判断屏初始化成功

屏初始化成功，一般呈现的现象是雪花屏。因为屏驱动里面，在 LCD_open_flow 中添加了 lcd_fb_black_screen 的注册，故正常情况下开机是有背光的黑屏画面。

6.7 黑屏-无背光

一般是电源或者 pwm 相关配置没有配置好。参考lcd_pwm开头的相关配置。

6.8 送图无显示

排除步骤:

1. 首先执行 spilcd 模块测试命令，如果能正常显示颜色画面，说明显示通路正常，只是应用未能正确配置送图接口。如果执行测试用例无法正常显示颜色画面，接着下面步骤。
2. 屏驱动里面，在 LCD_open_flow 中删除 lcd_fb_black_screen 的注册，启动后，如果屏初始化成功应该是花屏状态 (大部分屏如此)。
3. 如果屏没有初始化成功，请检查屏电源，复位脚状态。
4. 如果屏初始化成功，但是发数据时又没法显示，那么需要检查是不是帧率过快，查看帧率控制。
5. 如果电源复位脚正常，请检查配置，lcd_dbi_if, lcd_dbi_fmt是否正确，屏是否支持, 如果支持，在屏驱动里面是否有对应上。
6. 尝试修改lcd_dbi_clk_mode。

6.9 闪屏

非常有可能是速度跑太快，检查 SPI/DBI 的传输速率是否和 LCD 的驱动 IC 的速率匹配。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.10 画面偏移

画面随着数据的发送偏移越来越大。

尝试修改lcd_dbi_clk_mode。

深圳易新泰科技有限公司YiXinTai

7 总结

调试 LCD 显示屏实际上就是调试发送端芯片和接收端芯片 (LCD 屏上的 driver IC) 的一个过程：

1. 添加屏驱动请看编写屏驱动。
2. 仔细阅读屏手册以及 driver IC 手册 (有的话)。
3. 仔细阅读板级显示配置参数详解。
4. 确保 LCD 所需要的各路电源管脚正常。

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2023 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。